GitHub 三人协作操作手册

智能赛车项目 / Windows + GitHub Desktop / 零基础执行版

版本：v1.0；编制与官方文档核对日期：2026-10-11。

你们的仓库： AgrumeStly/yeye，私有。你们的角色：主控、视觉、硬件。你的台式机已有本地仓库，另外三台电脑需要首次下载。

先照做，再理解。 第一次用第4-5页设置；日常照第6-9页；交接照第11-12页；最后一页前的第20页可打印放在桌边。本文操作用 GitHub Desktop 和网页，日常无需输入 Git 命令。

页码	解决的问题	谁要看
2-3	常用词、界面按钮与流程	全员
4-5	安装、邀请、首次下载或接入	全员
6-8	新任务、提交、推送、继续开发	全员
9-10	换电脑、三人模块对接	全员
11-12	提交合并请求、检查和合并	全员 / 主控
13-14	管理任务、处理冲突	全员
15-16	撤销和常见故障	遇到问题时
17-18	稳定版本、工程与资料管理	全员
19-20	第一次练习、日常速查	全员
21	官方参考文档与教程维护	按需

现状说明： 仓库已有资料和视觉参考代码，尚不能据此认定整车程序可以运行。教程示例文件、分支、Issue、PR和版本标签是练习或建议，不代表已经创建。

按钮采用官方英文名称，后面解释用途；界面版本和窗口宽度可能影响位置，按名称寻找。教程中的界面图为示意，不是真实软件截图。

先理解：文件到底保存在哪里

02 / SMART CAR TEAM / GITHUB DESKTOP

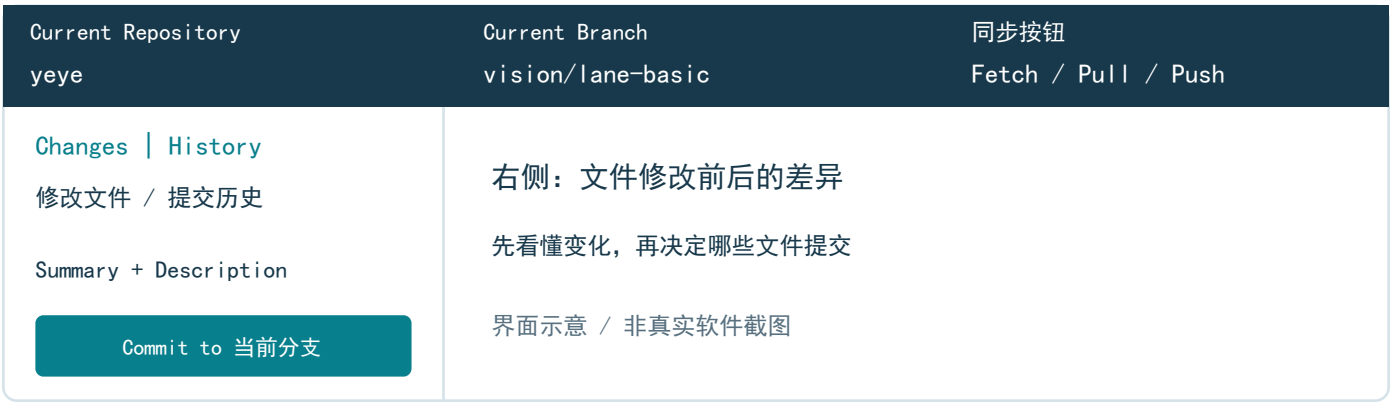
你们有四份独立的本地工程：主控台式机、主控笔记本、视觉电脑、硬件电脑。GitHub 上还有一份共享的远端仓库。每台电脑都可以离线改文件，联网同步后其他人才会得到。

工作文件 Save / 保存 > 本地历史 Commit / 提交 > 网页分支 Push / 上传 > 共享main PR / 合并

名称	中文理解	容易误会的地方
Repository / repo	整个项目及其版本历史	本地仓库与网页仓库是两份副本
Clone	第一次下载完整仓库	一台电脑一般只做一次
Branch	一条独立的开发路线	切换分支会改变同一文件夹里的内容
main	团队用于集成的主分支	任务分支推送后不会自动进入main
Commit	在本机留一个有说明的版本	保存文件后还要提交
Fetch	查询并下载远端的新版本信息	不等于已把新内容放进当前工作文件
Pull	把远端当前分支的更新合进本地	更新的是所选分支
Push	上传本地提交	同伴随后拉取才能看到
PR / Pull request	请求把任务分支合进main	此处的“Pull”不是Desktop的拉取按钮
Merge	合并两条路线的修改	没有冲突也不代表代码逻辑正确
Tag / Release	标记版本 / 发布版本说明与附件	不会自动烧录到单片机

例子： 视觉改了识别代码并按保存 → 只有自己的文件变了；Commit → 自己电脑有历史；Push → 网页任务分支能看到；PR合并 → 网页main能看到；主控Pull main → 主控工程里才有它。

认识 GitHub Desktop 的五个区域



示意图只展示用途。第一次先确认顶部仓库是 yeye，分支是你准备工作的分支；再看右侧同步按钮。

区域 / 按钮	用途	你们什么时候用
Current Repository	选择本地仓库	多个项目之间切换，确认选的是yeye
Current Branch	选择分支、创建分支	开工、换任务、接着另一台电脑工作
Fetch origin	检查远端是否有新提交	每次开工和需要同伴更新时
Pull origin	拉取当前分支更新	Fetch后显示有新提交时
Publish branch	第一次上传这条分支	任务分支第一次分享时
Push origin	上传本地尚未推送的提交	后续每次提交后、收工前
Changes	尚未提交的修改	看清哪些文件即将入库
History	已有提交历史	查版本、查看修改、做恢复
Summary / Description	提交标题 / 详细说明	写清本次改动及验证情况

正常按钮会变化： 新分支先显示Publish branch；上传后常见Fetch origin；发现远端更新可能显示Pull origin；本地有未上传提交会显示Push origin。没有某按钮时先检查分支和当前状态。

未提交修改是工作文件。 不要以为切换分支就能自动保管它；有修改时先提交，确认后再换分支。关掉IDE不能代替提交。

第一次准备：账户、安装与成员邀请

每个人用自己的 GitHub 账户。 主控的台式机和笔记本使用主控自己的账户；视觉、硬件各用各的账户，提交作者才能清楚。完成邮箱验证，保存好自己的登录方式。

主控：先把两位队友邀请进仓库

1. 打开 成员管理页，或从仓库进入 Settings → Collaborators。
2. 点 Add people，输入队友的准确 GitHub 用户名；核对后发送邀请。
3. 队友在通知或邀请邮件中点接受；未接受前，私有仓库可能显示404。
4. 队友登录后打开仓库，能看到目录即完成访问检查。默认工作方式是同一仓库内分支协作。

每台电脑：安装并登录 GitHub Desktop

1. 从 官方下载安装页 下载Windows版并安装。
2. 打开软件，通过 Sign in to GitHub.com 登录自己的账号；按浏览器提示完成授权并返回软件。已有安装可通过 File → Options → Accounts 检查账号。
3. File → Options → Git：Name填自己的作者名，Email选关联到自己GitHub账户的邮箱或该账户提供的noreply邮箱，保存。
4. 三人各自确认显示的账户正确。若浏览器登录与Desktop登录不同，分别检查两个地方。

完成标准： 队友能访问私有仓库；各电脑Desktop登录正确；作者姓名和邮箱属于实际操作者。此时先去第5页接入项目。

设备登录时，只输入自己电脑上刚生成的设备码。如果内置浏览器确认按钮不可用，可在常用浏览器完成同一次授权；不把密码或验证码发给队友。

依据：官方参考 [S1] [S2] [S3]，链接见第21页。

第一次把项目放到四台电脑

05 / SMART CAR TEAM / GITHUB DESKTOP

A / 你的台式机：添加现有仓库

1. GitHub Desktop → File → Add local repository。
2. 选择 D:\Projects\yeye，点 Add repository。不要选择它的control或vision子文件夹。
3. 顶部确认yeye、main；用 Repository → Show in Explorer 打开目录，确认就是原工作目录。
4. 点 Fetch origin；如出现Pull origin再点。原工程已连接GitHub，无需新建仓库或重复上传。

B / 你的笔记本和两位队友：首次Clone

1. File → Clone repository → URL。
2. Repository URL输入：https://github.com/AgrumeStly/yeye.git。
3. Local Path指定一个新的、空的工程目录，例如D:\Projects\yeye。软件可能按选择补上仓库名，以对话框最终路径为准。
4. 点Clone；完成后检查README、competition、control、vision等目录。把IDE打开位置设为这个实际目录。

不要Clone到已有一堆文件的目录，也不要把Download ZIP当日常协作工程。Clone失败先查登录及邀请状态，不要再创建一个同名远端仓库。

下载完成后：验证“真的能调试”

安装对应IDE、编译器、芯片支持包、下载器和串口驱动；版本由模块负责人记录到README。Git只传文件，不安装这些工具。现在主控目录尚无完整工程，先验证目录和下载器连接；有首版程序后，再从Clone目录编译、烧录最小演示。

完成标准： 四台电脑打开同一个远端项目，各自有独立本地目录；桌面快捷方式和IDE工程路径没有指向旧副本。三人不要另建各自的同名项目仓库。

依据：官方参考 [S4] [S5]。

每次新任务：先同步，再开分支

适用：刚开始一个新功能、修一个新问题或写一份新资料。若今天继续昨天的任务，直接按第8页操作。

开工前的操作

1. 保存并关闭可能自动改写工程文件的IDE窗口。打开Desktop，选择yeye，查看Changes。
2. 若有昨天的未提交修改，先按第7页提交并推送原任务分支；不要随意丢弃。
3. Current Branch选择main；点Fetch origin，若出现Pull origin就点。确认main已经同步。
4. 先商量今天只完成哪一件事；在Issue里记录主责和完成标准，方法见第13页。
5. Current Branch → New Branch，输入任务名；若出现基于哪个分支的选项，选main；点Create Branch。
6. 顶部确认新分支名，再用IDE修改这个本地目录里的文件。完成第一个提交后按第7页上传分支。

方向	示例分支名	一次任务范围
主控	control/motor-stop	电机启停及悬空验证
视觉	vision/lane-basic	边界识别的第一版
硬件	hardware/wiring-v1	接线图与引脚核对
整车修复	fix/uart-timeout	通信超时停车
文档	docs/github-practice-control	主控的入门练习

分支用英文、小写、短横线；任务完成合并后，下一任务从最新main再开新分支。不要把control、vision、hardware永久当三份总版本；每个方向可以有多条任务分支。

完成标准： 正在编辑的目录来自Desktop；Current Branch是当前任务名；这个任务的目标和负责人已明确。

依据：官方参考 [S6]；任务命名和分工为本团队建议。

修改后与收工前：提交并上传

从文件修改到网页能看到

1. 在IDE或编辑器中保存全部文件，先做这次改动所需的验证：编译、板上演示、图像回放或接线复核。
2. 回到Desktop → Changes；逐个点文件，看差异。文字差异中绿色通常为增加，红色为删除；核对是否只有本任务相关内容。
3. 勾选要提交的文件。若有编译输出、临时图片、私人文件或与任务无关的修改，先取消勾选并处理原因。
4. Summary写具体标题，例如“主控：添加通信超时停车”。Description写怎么验证、是否已上车、还有什么问题。
5. 点 Commit to 当前分支。这一步只记到本机；看History确认新提交出现。
6. 第一次上传这条分支点Publish branch；以后点Push origin。等操作结束，确认没有未推送提示。
7. 网页打开仓库，切到同名分支；检查最新提交标题和关键文件。让队友看时发送分支名或PR链接。

每次收工的三项检查

文件已保存；要保留的修改已提交；提交已推送。 Changes为空只说明没有未提交的修改，不能证明已经上传。若还有需要保留的修改，不要仅靠截图或聊天记录交接。

任务还没做完也能保存。 在任务分支提交“阶段保存：串口接收已通，停车待测”，并推送；不要合入main。必要时创建Draft PR并说明阻塞。建议每完成一个小成果提交一次，收工前必推送。

测试记录要与代码一起交付：例如testing/2026-10-18_run01/记录.md，写程序提交号、参数、接线版、操作、结果。没测试的部分写“未测试”，不要填“已通过”。

依据：官方参考 [S7] [S8]；记录格式为团队建议。

第二天继续同一任务：分清两种更新

日常继续：拉取这条任务分支

1. 检查Changes；若有本地修改，先保留并提交到它所属的分支。
2. Current Branch选择昨天的任务分支，例如vision/lane-basic。
3. 点Fetch origin；有Pull origin时拉取，然后打开IDE继续修改。
4. 开发、验证、Commit、Push，重复第7页。

这只更新同名任务分支。 即使网页main已经合入队友的新代码，你的任务分支也不一定含有它。

需要最新main的接口或代码：合入任务分支

1. 先Commit并Push任务分支，确认工作目录没有未提交修改。
2. 切main，Fetch并Pull，使本地main同步。
3. 切回原任务分支，打开Current Branch → Choose a branch to merge into 当前分支。
4. 选择main，核对方向是 main → 任务分支，再点Merge。不是把未完成任务合进main。
5. 没冲突则按需验证并Push；有冲突按第14页和主控一起处理。

什么时候需要合main： 队友修改了你依赖的协议、头文件、配置或算法接口，或准备提交PR前要适配最新集成代码。单纯自己继续编辑同一模块，不必为了“更新”反复合并。

分支切换提醒： 软件提示Leave my changes或Bring my changes，说明有未提交文件。新手先取消切换、看清Changes，提交到正确任务分支后再切；不要不看提示就把修改带进main。

依据：官方参考 [S8]；需要更新main的时机为团队建议。

台式机与笔记本：同一任务怎么接着做

09 / SMART CAR TEAM / GITHUB DESKTOP

你的两台电脑使用同一账户，但代码不会自动同步。先确认相同仓库，再确认相同分支，最后确认相同提交。 一次只在一台电脑推进同一任务。

离开台式机之前

1. 在当前任务分支保存、检查、Commit、Push。
2. 记录分支名和最新提交号的前7位，如control/motor-stop、abc1234；可在History或网页提交链接查看。
3. 确认当天必要样本、参数、接线说明都在仓库，或笔记本已下载共享存储的附件。
4. 出门前在笔记本完成下面步骤，避免到现场才发现没有网络或缺驱动。

笔记本接手

1. 打开Desktop，选yeye，点Fetch origin。
2. Current Branch搜索完整的任务分支名，选择远端已发布的同名分支；如果已在本地，直接切换。
3. 有Pull origin就点，查看History确认最新提交与台式机记录相同。
4. 用IDE打开Clone目录，检查本机编译/烧录配置；先编译，再上板验证。
5. 调试结束后把参数、问题记录和代码Commit、Push；回到台式机同分支Fetch/Pull。

离线现场： 可以修改和Commit，但Push要联网。离线时不要让台式机同时修改同一分支；恢复网络后先Fetch，若发现远端也变了，按第16页处理推送失败。

分支合并后换电脑： 两台电脑都切main并Pull，再从新main建下一任务分支。不要在已经结束的旧分支继续下一项任务。

不能靠账号判断版本。 两台电脑登录同一账户，不代表它们工作目录一致；应对照仓库、分支和提交号。不要通过网盘同步整个工作工程或.git目录。

三个人怎么交接代码与接口

方向	主责目录	每次交接最少提供
主控	control/	工程、依赖、控制输入输出、参数、验证结果
视觉	vision/	脚本、输入样本、路径输出、性能与有效标志
硬件	hardware_preparation/	接线图、引脚表、实物型号、照片与检查结果
三人共同	testing/	程序版本、接线版、试跑数据、故障与复测

共同接口只认一份文档。 建议后续新建project_planning/interfaces.md，主控与视觉共同审核，硬件复核接线。不要把关键协议只放在群聊里。

接口至少写：通信电平和引脚、串口参数、包格式、字节顺序、字段单位、正负方向、无效标志、超时停车、命令ID、协议版本。修改接口时单开任务分支，通知双方一起调整。

一个完整交接例子

1. 硬件先在hardware/wiring-v1提交实物接线，PR写清串口、电机与编码器接口，主控复核。
2. 视觉在vision/lane-basic提交脚本和样例结果，注明偏差符号与无效状态；主控用模拟结果包调解析。
3. 主控检查两个PR，先合接线与约定，再合视觉实现；三人Pull main后统一实物配置联调。
4. 实车测试发现问题，记录对应提交号，新开fix分支修复，保留失败图片和日志；修复验证后再合main。

职责不是文件锁。 两人可以改同一仓库；同时改同一份文件更容易冲突，开工前先说清。PDF、Excel等二进制文件不能像代码一样逐行合并，由一人生成，另一人提修改意见。

ChatGPT辅助生成的代码或说明，也要由模块负责人理解和验证后再提交。队友通过仓库拿到成果；讨论问题时提供文件路径、分支、提交号、复现步骤和日志。

任务完成：作者怎么发 PR

PR是交付和讨论入口。准备合进main时使用；只是收工保存时，Commit和Push就够了。

在Desktop和网页创建

1. 当前任务分支先保存、Commit、Push；确认网页同名分支有最新提交。
2. Desktop点Preview Pull Request，检查变动；选择Create Pull Request进入网页。也可以直接在仓库网页Pull requests → New pull request。
3. 确认 base是main，compare是自己的任务分支。看清 Files changed，避免夹带别人的任务或缓存。
4. 写标题和交付说明。可供验收时点Create pull request；未完成但需要讨论时选择Draft。
5. 把PR链接发给主控，说明需要他检查哪一项。有可选Reviewers时选择相应成员。

交付说明模板：直接按这个填

标题：视觉：添加基础边界识别
任务：关联Issue #实际编号
修改：vision/src/...；路径字段与协议版本
验证：使用哪些样本/板卡，操作和结果
限制：未覆盖的光照、尚未进行的实车测试
主控配合：接收哪个字段；如何运行/烧录
证据：测试记录路径；视频链接

修改PR里的问题时，继续在 同一任务分支 改、Commit、Push，PR会更新；不必重新创建一个PR。更新后留言说明补做的测试和对应提交。

完成标准： 网页PR处于Open状态，分支方向正确，变动范围清楚，测试说明能让主控复现。仅有“代码完成”四个字，不足以交接。

关联Issue时，尚未达到验收标准可写“关联 #编号”；确认PR合并即代表该任务完成时，可写Closes #编号，让默认分支合并后自动关闭对应Issue。

依据：官方参考 [S9]；模板为团队建议。

主控检查 PR：下载、测试、合并

建议主控作为集成人；自己的PR由视觉或硬件至少检查一遍。这是一项团队约定，当前没有据此自动启用分支保护或要求付费功能。

每个 PR 都走一次

1. 网页Pull requests打开PR，读交付说明；Files changed查看修改，核对接口、配置与测试记录。
2. 要本机试跑时，先提交自己的未保存任务；Desktop Fetch后切到该PR的任务分支，有更新就Pull。
3. 按作者步骤编译、回放或上板测试。单模块检查通过后，确认必要的整车回归；不能把“无冲突”当“无错误”。
4. 问题通过PR评论说明文件、表现和复现步骤。正式Review可在Files changed → Review changes中选择Comment、Approve或Request changes。
5. 作者更新后重新查看新提交，并复测受影响的部分。自写PR不能由自己Approve，找另一人检查。
6. 可以集成时，在PR的Conversation页点Merge pull request，再点Confirm merge；若下拉可选方式，本教程采用普通Create a merge commit。
7. 核对PR显示Merged，网页main包含结果；确认该分支不再使用后再删除远端任务分支。
8. 全员Desktop切main，Fetch/Pull，确认合并后的提交；从新main开始下一任务。

保留任务分支到测试完成。 如果无法合并，先看是冲突、权限还是仓库规则；冲突见第14页。教程不会要求你绕过仓库规则。

合并后不是自动部署。 主控要编译/烧录MCU，视觉要部署相应脚本，硬件要按约定接线。最后在testing/记录实际运行的版本组合。

依据：官方参考 [S10] [S11]；验证流程为团队建议。

用 Issue 管理任务和故障

Issue就是“有编号的待办单”。代码修改通过PR交付；接线、测试和报名等没有代码的任务，也可以通过Issue跟踪。

开一项任务

1. 仓库网页Issues → New issue；若有模板选合适模板，否则开空白任务。
2. 标题写具体目标，例如“主控：串口断连后车辆受控停车”。
3. 填责任方向、截止日期、完成标准、依赖与当前阻塞；Assignees选择已加入仓库的实际负责人。
4. 点提交任务的按钮。截止日期先写正文即可，不必建立复杂看板。
5. 开发期间在同一Issue更新结果。模块任务的PR写Issue编号；硬件任务直接贴记录与照片，由同伴确认后Close issue。

> 负责人：主控（选择实际成员为Assignee）
> 截止：2026-10-25 21:00
> 目标：识别串口超时，减速并停止电机
> 依赖：视觉提供有效标志与更新间隔
> 完成标准：悬空断连测试通过；阈值已记录
> 证据：testing/.../record.md；PR #实际编号
> 状态：进行中；阻塞：编码器极性待复核

报告故障时的五项信息

版本、操作、预期、实际、证据。 例如：“提交abc1234，接线v2，悬空转轮后拔串口；预期停车，实际左轮继续转；附串口日志和视频链接”。这比“程序有问题”更容易定位。

三人每个工作日开始看自己负责的Issue，结束更新；主控每周统一核对未完成事项。完成要有证据；合并代码后仍未实车验收的任务保持Open，避免提前Closes。

仓库保持私有；上传日志和照片前看清是否包含账号密钥、私人信息或与项目无关的文件。

依据：官方参考 [S9] [S12]；任务模板为团队建议。

出现冲突：共同决定保留什么

冲突表示两条路线改到了同一处，Git无法自动选择。它没有替你决定程序逻辑，也不意味着已经丢失代码。先保留现状，确认冲突文件和两边的目的。

简单文本冲突：在 PR 网页解决

1. PR中点Resolve conflicts。若按钮不可用，交由主控在本地处理，不要为此强推覆盖。
2. 与另一位作者核对冲突两侧分别想实现什么；手动整理出一份最终内容，删除冲突标记。
3. 每个文件检查全部冲突，点Mark as resolved；全部处理后Commit merge。这是把base的内容合入任务分支，PR尚未因此合入main。
4. 作者Desktop切任务分支，Fetch/Pull，把网页处理结果拉回本机，编译或执行对应验证。
5. 更新PR测试记录，主控重新检查，通过后才按第12页合并。

冲突标记示例：仅用于认识结构

```
<<<<<<< HEAD
UART_TIMEOUT_MS = 200
=======
UART_TIMEOUT_MS = 300
>>>>>>> main
```

示例里两侧都改了停车超时。主控与视觉应根据实测更新间隔共同确定阈值，最后只留下确认后的赋值。不能把两行都保留并靠“最后一行生效”解决。HEAD/current指当前被合入的工作上下文，incoming指另一侧；不要把它们固定理解为“我的/队友的”。

本地合并有冲突时，用Desktop列出的文件在编辑器中处理；不清楚逻辑先停止并联系同伴。解决后按软件提示继续合并，验证并Push。PDF、Excel的冲突应由负责人选定或重新生成完整版本。

依据：官方参考 [S13]；参数数值仅为演示，不是车辆推荐值。

改错了：按发生阶段选择恢复方法

恢复前先确认仓库和分支；重要的未提交修改先另存副本。不要把“回到旧代码”理解为删掉后面的所有团队历史。

发生在哪一步	推荐处理	结果
保存了，还没Commit	手动改回，或只Discard明确不要的文件	当前未提交修改被移除
刚Commit，还没Push	对最后一个未推送提交使用Undo	修改回到Changes，可修正后再提交
已Push到任务分支	新提交修正；必要时Revert错误提交	保留旧历史，新增修复或反向提交
PR已合进main	开fix分支修复；需要回退时主控处理	通过新PR恢复，保留团队历史

两个常用按钮

Undo： Desktop在Changes页底部提供Undo时可撤销最近尚未推送的提交；文件修改仍保留。看Changes、修正、重新Commit，再Push。不要把没有Undo按钮当丢失代码。

Revert Changes in Commit： History中右键目标提交使用；产生一个抵消该提交的新版本，原提交仍在历史里。用于已推送修改时，比强行重写历史更容易协作。

若要撤销已合并进main的一个PR，先由主控在该PR页面查找Revert。可用时会生成回退PR，仍需检查和测试后合并；按钮不可用或回退范围复杂时，开fix分支手动恢复所需代码，不对合并提交盲目操作。

Discard Changes会移除工作文件中的修改。 Desktop通常将弃置内容放到回收站，清空前可能恢复，但不要把这当可靠备份；有用修改先另存。Force push、Reset、Rebase不是你们日常恢复流程。

依据：官方参考 [S7] [S14] [S15] [S16]。

常见故障：先看状态，再做下一步

看到的现象	常见原因	先这样处理
私有仓库404 / 无法Clone	未登录、未接受邀请、地址错误	登录正确账号，确认邀请已接受，再核对URL
队友说上传了，我看不到	只Commit未Push；查看了别的分支	让对方发分支和提交号；Fetch后切同名分支并Pull
Fetch后文件没更新	还没Pull或当前分支不对	核对分支，出现Pull origin再点
切分支提示有Changes	当前还有未提交修改	取消切换，先看清并提交到正确分支
Push失败 / 远端有新提交	另一台电脑或成员更新了同一分支	先Commit保留本地，Fetch/Pull，解决冲突后再Push
提示要Fork	对原仓库没有写权限	不急着创建Fork，先让主控检查成员访问
列出大量编译/临时文件	忽略规则未覆盖，或文件已被跟踪	不全选提交；主控补规则并处理已跟踪文件
网络断开	无法访问远端	继续本机Commit，记下未推送状态，联网后同步
没有Push按钮	可能没有未上传提交或还需发布分支	先查History、分支及Publish branch，不重复建仓库

不小心在main改了文件怎么办

尚未Commit： 先另存关键修改。Desktop创建任务分支时若提示带走修改，核对后选择Bring my changes到新分支；顶部分支正确后再提交。提示不清楚就先取消，让主控协助。

已在main Commit但未Push： 可先Undo最近那个未推送提交，让修改回到Changes，再按上一步转入任务分支。多个提交或已经Push时，不继续乱点Undo，由主控安排修正及同步。

发生问题时发给主控： Desktop提示的原文、Current Branch、最新提交号、Changes是否为空、是否已经Push。不要发账号密码、访问令牌或设备验证码。

界面提示是排查入口；有冲突看第14页，有撤销需求看第15页。不要用复制覆盖整个项目的方法“同步”。

稳定能跑后：做一个可恢复的版本

一次整车版本要绑定整套文件。 主控程序、视觉/语音脚本、参数、协议和接线都匹配，才适合作为恢复点。代码能编译不等于已经跑完一圈。

主控建立版本标签

1. 确认需要的PR已合并；Desktop切main并Fetch/Pull。
2. 记录即将验证的main提交号；三人按同一版本部署到板上，完成约定跑圈与功能测试。
3. 把测试记录提交并合入main，说明实际运行的是哪个代码提交；若main同时又有代码变化，先重新测试或选择此前明确验证的提交，不给未测试代码贴稳定标签。
4. Desktop → History，右键选定的提交 → Create Tag，输入例如v0.1-low-speed，确认创建。
5. 按Desktop提示Push，随后在网页Tags确认标签存在、指向正确提交。保留标签，不把同名标签改指另一版。

可选：网页创建 Release，方便现场取用

仓库Releases → Draft a new release → Choose a tag选择已有验证标签。标题写“低速完赛验证版”，正文写接线版、功能、圈速/成功率、已知问题和恢复步骤；附件放对应固件、脚本或参数包，点Publish release。

源代码归仓库，发布附件应由同一版本生成；电池、安装和现场条件仍需复核。仓库中目前没有已验证整车版本，本页标签名称都是示例。

需要旧版临时调试

主控在History定位标签对应提交，右键Create Branch from Commit，起名debug/v01-check；切到该分支后重新编译/部署整套文件。这个分支不自动影响main。随后根据发现的问题从最新main开fix分支修复，避免把整条旧分支直接合回main。

依据：官方参考 [S17] [S18]；发布时机为团队建议。

什么文件入库，以及如何保持工程可运行

文件类别	放置方式	说明
MCU工程、源代码、必要工程配置	control/	保留工程文件、链接脚本和生成配置，不能只交.c文件
视觉/语音脚本、必要模型与样本	vision/	写清部署位置、依赖和可复现输入
接线、器件、装配照片	hardware_preparation/	生成脚本放source/，旧版放archive/
规则、任务、接口约定	competition/、project_planning/	团队与官方资料分开放
测试日志、小型样例	testing/	写明提交、参数、硬件版和结果
大视频和大量原始样本	共享存储	仓库记录链接及用途；现场前下载所需文件
临时预览、编译缓存、本机配置	本地保留，Git忽略	当前.gitignore已覆盖部分常见情况

每个模块的README至少写五项

1. 用哪块板、接线在哪里；需要什么固件和IDE/工具版本。
2. 从哪个工程文件打开，怎么编译、下载或运行。
3. 外部依赖如何获得；工程里尽量使用相对路径。
4. 接收/发送什么数据，参数在哪里改。
5. 一个最小验证操作，以及预期结果和已知限制。

重要例外： 被.gitignore忽略的文件，可能是工程真正依赖的文件；新芯片库如果放在名为Release的目录，不应因为目录名被忽略就丢掉。每次在新电脑做一次完整编译，才能发现缺件。

.gitignore不会自动移除已经入库的文件；遗漏由主控统一处理。正式烧录用的固件可作为Release附件，日常编译缓存通常不提交。PDF、Excel指定一名编辑者；文档有生成脚本时改源文件后再生成。

不把API密钥、个人访问令牌或密码提交到项目。项目代码通过Git同步；本机串口号、下载器连接、驱动和IDE授权仍由每台电脑自行配置。

第一次练习：三人各交一份小文档

建议约60-90分钟集中练习，主控在旁协助。用入门文档验证协作链路，避免第一次同时调复杂驱动。

三人分别执行

1. 按第4-5页完成账户、邀请及Desktop接入；确认能访问私有仓库。
2. 同步main，各自开分支：docs/github-practice-control、docs/github-practice-vision、docs/github-practice-hardware。
3. 在自己目录下新建不同文件：control/github_practice.md、vision/github_practice.md、hardware_preparation/github_practice.md。
4. 文件写三行：我的负责方向；本机使用的工具；下一项具体任务。保存后在Changes核对，只勾选自己的文件。
5. Commit、Publish/Push，网页切同名分支验证。各自按第11页创建PR。
6. 主控按第12页检查并依次合并；主控自己的PR由另一人检查。全员切main并Pull，应能看到三份文件。
7. 主控在台式机新开docs/laptop-handoff，在练习文档添加一行并推送；笔记本切同名分支拉取，再补一行并推送，台式机拉取验证。

练习通过标准： 三人都能自己提交、上传和发PR；全员拿到三份main文件；台式机与笔记本接力成功；没有通过复制整个文件夹完成交接。

五个自测题

1. Commit之后，队友立刻能看到吗？
2. Fetch之后，本地文件一定更新了吗？
3. 任务分支Push之后，main自动更新了吗？
4. 笔记本和台式机同账号，工作目录一定一致吗？
5. 冲突两侧都保留，是否一定正确？

答案：都不是。分别还需要Push与队友Pull、必要的Pull、PR合并、同分支同步并核对提交、按程序逻辑整理最终内容。

这套练习是教程方案，文件和分支尚未替你们创建。练完后再按实际硬件任务开始开发。

每次照着做：一页速查

开始一个新任务

确认yeye → 处理已有Changes → 切main → Fetch/Pull → New Branch → 确认任务名 → 开发。

继续昨天的任务

确认yeye → 处理已有Changes → 切原任务分支 → Fetch/Pull → 开发。 要最新main接口时，按第8页合main进任务分支。

每次保存成果 / 收工

保存文件 → 验证 → 看Changes → 勾相关文件 → Summary → Commit → Publish/Push → 确认网页分支和提交。

任务交给主控

推送完成 → 发PR：base main、compare任务分支 → 写变更与测试 → 主控检查 → 修改后同分支再Push。

主控合并 / 全员接收

读PR → 看差异 → 下载任务分支测试 → Merge并确认Merged → 全员切main → Fetch/Pull → 验证部署。

换电脑

原电脑Commit/Push → 记录分支与提交 → 新电脑Fetch → 切同名分支 → Pull → 核对History → 编译调试。

三人收工检查

检查项	合格标志
选对地方	仓库yeye；任务分支名称正确；IDE打开对应目录
留下成果	所需修改已Commit；测试记录和参数有保存
上传到位	已Publish/Push；网页同名分支显示提交
接口交接	PR或Issue注明协议、依赖和未完成项
现场可恢复	记录整车版本；必要文件、驱动、样本已准备

遇到问题： 冲突看14页，撤销看15页，登录/同步异常看16页。先说明分支、提交和提示，不复制覆盖，不用Force push“解决”。

官方参考与教程维护

以下官方操作文档于2026-10-11核对。本文用中文重新组织为你们的工作流程，界面示意由本教程绘制；三人分工、验收和命名属于团队建议。按钮调整时按对应官方页面查找。

- S1 安装与初次设置
- S2 配置提交作者
- S3 邀请成员
- S4 添加本地仓库
- S5 下载仓库
- S6 创建、发布与切换分支
- S7 查看修改、提交与弃置
- S8 分支同步与合并
- S9 创建Issue与PR
- S10 检查PR
- S11 合并PR
- S12 Issue与任务管理
- S13 处理网页冲突
- S14 Undo未推送提交
- S15 Revert已提交修改
- S16 回退一个PR
- S17 创建标签
- S18 发布Release

仓库地址：https://github.com/AgrumeStly/yeye。被邀请的成员登录后才能访问。

教程保存于project_planning/；Markdown是可编辑内容源，source/generate_github_guide.py从它生成PDF。更新时先修改Markdown，再重新生成并检查PDF，提交相关文件。

本教程没有执行成员邀请、练习任务或功能开发，也不把main自动设置为受保护分支。实际配置变更由团队按需要实施。